p

```
import random

# Algorithm configuration parameters
ALPHA = 1.0  # Pheromone history importance
BETA = 2.0   # Current road condition importance
RHO = 0.1    # Evaporation rate (10%)
Q = 100.0    # Pheromone reward constant

class Road:
    def __init__(self, road_id, length, speed_limit):
        self.road_id = road_id
        self.length = length
        self.speed_limit = speed_limit
        self.pheromone = 1.0  # Baseline initial pheromone
        self.current_vehicle_count = 0
        self.recent_travel_times = []

    def get_visibility(self):
        """Favors shorter, high-speed, and empty roads."""
        base_speed_factor = self.speed_limit / self.length
        density_factor = 1.0 / (self.current_vehicle_count + 1)
        return base_speed_factor * density_factor

class Intersection:
    def __init__(self, intersection_id):
        self.intersection_id = intersection_id
        self.outgoing_roads = {}  # Format: {destination_node: Road_Object}

    def add_connection(self, destination, road):
        self.outgoing_roads[destination] = road

class Vehicle:
    def __init__(self, vehicle_id, start_node, end_node):
        self.vehicle_id = vehicle_id
        self.current_node = start_node
        self.destination = end_node
        self.path_taken = []
        self.current_segment_travel_time = 0.0

# --- SIMULATION RUN ---

# 1. Setup Map Graph (Intersections & Roads)
node_A = Intersection("A")
node_B = Intersection("B")
node_C = Intersection("C")

road_AB1 = Road("Highway_AB", length=10.0, speed_limit=60.0)
road_AB2 = Road("LocalStreet_AB", length=5.0, speed_limit=30.0)
road_BC = Road("MainSt_BC", length=8.0, speed_limit=45.0)

node_A.add_connection(node_B, road_AB1)
node_A.add_connection(node_B, road_AB2)
node_B.add_connection(node_C, road_BC)

all_roads = [road_AB1, road_AB2, road_BC]

# 2. Deploy Vehicles (Ants)
active_vehicles = [
    Vehicle("Car_1", node_A, node_C),
    Vehicle("Car_2", node_A, node_C),
    Vehicle("Car_3", node_A, node_C)
]

print("--- Starting Simulation Step ---")

# Phase A: Probabilistic Routing & Vehicle Movement
for vehicle in active_vehicles:
    if vehicle.current_node == vehicle.destination:
        continue

    current = vehicle.current_node
    choices = current.outgoing_roads

    # Calculate probability scores for each path choice
    scores = {}
    total_score = 0.0
    for next_node, road in choices.items():
        score = (road.pheromone ** ALPHA) * (road.get_visibility() ** BETA)
        scores[next_node] = score
```

```
        total_score += score

    # Roulette Wheel Selection
    target = random.uniform(0, total_score)
    current_sum = 0.0
    selected_next_node = None
    chosen_road = None

    for next_node, road in choices.items():
        current_sum += scores[next_node]
        if current_sum >= target:
            selected_next_node = next_node
            chosen_road = road
            break

    # Execute movement onto the chosen road segment
    if chosen_road:
        chosen_road.current_vehicle_count += 1
        vehicle.path_taken.append(chosen_road)
        vehicle.current_node = selected_next_node

        # Calculate dynamic travel time affected by active traffic density
        traffic_multiplier = chosen_road.current_vehicle_count
        vehicle.current_segment_travel_time = chosen_road.length / (chosen_road.speed_limit / traffic_multiplier)
        chosen_road.recent_travel_times.append(vehicle.current_segment_travel_time)

        print(f"{vehicle.vehicle_id} chose {chosen_road.road_id}. Travel Time: {vehicle.current_segment_travel_time:.2f} min

# Phase B: Global Pheromone Evaporation
for road in all_roads:
    road.pheromone *= (1 - RHO)

# Phase C: Pheromone Update (Rewarding Clear Routes)
print("\n--- Map Pheromone Update Result ---")
for road in all_roads:
    deposit_amount = sum(Q / t_time for t_time in road.recent_travel_times)
    road.pheromone += deposit_amount

    # Flush slice metrics ready for next iteration loop
    road.recent_travel_times.clear()
    road.current_vehicle_count = 0

    print(f"Road {road.road_id} -> New Pheromone Level: {road.pheromone:.4f}")
```

```
--- Starting Simulation Step ---
Car_1 chose LocalStreet_AB. Travel Time: 0.17 mins.
Car_2 chose LocalStreet_AB. Travel Time: 0.33 mins.
Car_3 chose LocalStreet_AB. Travel Time: 0.50 mins.

--- Map Pheromone Update Result ---
Road Highway_AB -> New Pheromone Level: 0.9000
Road LocalStreet_AB -> New Pheromone Level: 1100.9000
Road MainSt_BC -> New Pheromone Level: 0.9000
```